



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC3413 — Implementación de Sistemas de Base de Datos — 1' 2026

LABORATORIO 4

Laboratorio: Transacciones y niveles de aislamiento.
Publicación: Miércoles 3 de junio.
Ayudantías: En clases.
Entrega: **Lunes 22 de junio hasta las 23:59 horas (cupones hasta miercoles 24).**

Descripción del laboratorio

En este laboratorio exploraremos manejo de transacciones en IIC3413DB [1]. Por ahora IIC3413DB soporta transacciones bajo dos niveles de aislamiento: `READ COMMITTED` (default) y `SNAPSHOT ISOLATION`; lo último asegurando un nivel similar como `REPEATABLE READ`. Las transacciones son implementadas usando MVCC, lo que ya exploramos en el Lab1 y strict 2PL, cual nos asegura manejo correcto de conflictos write-write. Componentes del IIC3413DB involucradas en el manejo de transacciones son:

- **TransactionManager**, cual maneja la lógica de la operación de una transacción: desde su creación, manejo de su write set (los objetos que transacción modifica), commits y aborts.
- **LockManager**, cual maneja la lógica de adquisición de los locks por la transacción.
- **HeapFile**, cual implementa la operación concreta de la transacción bajo la lógica de su nivel de aislamiento.

Otras clases importantes son la clase **Transaction**, cual guarda los datos de la transacción y TxID (el número identificando a una transacción; siempre estrictamente mayor a 0). También, para manejar cómo el usuario trabaja con las transacciones, nuestro sistema implementa una arquitectura cliente-servidor cual en su **Session** permite lanzar una transacción. Las otras componentes cuales interactúan con las transacciones son **UpdateIter** y **DeletionIter**, cuales permiten actualizaciones de la base de datos. En general cada operación que permite revisar o modificar algo en la base de datos estará interactuando con transacciones.

En este laboratorio ustedes solo trabajarán con la clase **HeapFile**, pero para tener una idea del flujo de transacciones en un motor relacional, a continuación explicaremos algunos detalles de las componentes principales. Igualmente, en `transactions.md` les dejamos un resumen breve.

MVCC y transacciones en IIC3413DB

Suposiciones fundamentales de transacciones:

- Cada transacción en IIC3413DB recibe un TxID único; quiere decir un número mayor o igual a 1. Esto será su identificador durante toda su vida útil.
- TxID se asigna de manera creciente. Si $TxID_1 > TxID_2$, la primera transacción empezó después de la segunda.

- Cada transacción opera sobre una cadena de las versiones de la tupla en el heap file (MVCC).
- Cuando la transacción hace **ABORT**, limpia todas las versiones creada por si misma en el heap file y las marca **físicamente eliminadas** (ver abajo). Esto quiere decir que todo que una transacción puede ver en el heap file son tuplas con **COMMIT** o cuales están modificadas por una transacción en curso. Tuplas de una transacción que hizo **ABORT** son siempre invisibles.
- Una transacción tiene su **snapshot** – un conjunto de TxID de las transacciones activas cuando la transacción comenzó (solo en **SNAPSHOT ISOLATION**).
- Una transacción tiene su *write-set* – el campo **first_rids** cual guarda todas las tuplas modificadas por esta transacción (las identifica cómo un par (**tableId**,**RID**)).
- La transacción tienen un **TransactionState** cual puedes ser **ACTIVE**, si la transacción está en curso, o **COMMITTED**, si la transacción terminó (incluso si hizo abort).

Cada record en el *heap file* es una **versión de tupla** (recordar Lab1). Cada tupla física almacena un header de la tupla junto con los datos de las columnas. El header contiene:

Campo	Tipo	Significado
t_min	uint64_t	tx_id que creó esta versión.
t_max	uint64_t	tx_id que reemplazó esta versión; 0 = aún activa (<i>live</i>).
next	RID	RID de la siguiente versión (más nueva) ; RID(-1,-1) = eliminación lógica .
first	RID	RID de la cabeza de la cadena — es idéntico para todas las versiones en una cadena.

Cadena de versiones (Version chain): Cuando se actualiza una fila, se inserta a heap file una nueva tupla con **t_min** = **tx_id** y **first** = **original.RID**. La versión antigua obtiene **t_max** = **tx_id** y **next** = **new.RID**. La cadena tiene un enlace simple hacia adelante (desde la más antigua hacia la más nueva) mediante el campo **next**; todas las versiones comparten el mismo puntero **first**.

Eliminación lógica (Logical deletion): Marca la última versión con **t_max** = **tx_id** y **next** = **RID(-1,-1)**, marcando el final de la cadena como eliminado sin borrar los bytes correspondientes a los datos. Esto ocurre cuando una transacción elimina la tupla durante su ejecución y hace **COMMIT**.

Eliminación física (Physical deletion): Escribe un -2 en el slot de la página (identificada por **RID**). Se utiliza únicamente durante el proceso de **ABORT** y *vacuum*, nunca de forma directa por una operación de escritura activa.

UNDO (Clean-up on abort): Es **inmediata y síncrona** — **undo()** se ejecuta como parte del proceso de abort antes de que se liberen los *locks*, por lo que las versiones abortadas nunca son visibles para ninguna otra transacción. No quedan entradas “abortadas” olvidadas en el *heap*.

El **tx_id** 0 es un centinela reservado: **t_max** == 0 significa que “esta versión nunca ha sido reemplazada”.

¿Cuándo una transacción puede ver una versión de la tupla?

Cómo ya saben, dependiendo de su nivel de aislamiento, una transacción puede tener una visión diferente de las tuplas en el heap file. Pro ejemplo, una transacción Tx corriendo bajo **READ COMMITTED** ve tuplas creadas por una transacción que empezó después de Tx y hizo **COMMIT**, mientras una transacción corriendo bajo **SNAPSHOT ISOLATION** no las ve. Aquí explicaremos las regla de visibilidad para los dos niveles de aislamiento en IIC3413DB.

Para acceder a las tuplas visibles a nuestra transacción usamos el **HeapFileIter**:

- Aquí uno itera sobre todos los **RIDs** del heap file cuales no fueron físicamente eliminados.

- Cuando el RID representa una primera versión de la tupla, se llama el método `HeapFile::read` cual retorna la versión de la tupla en esta cadena visible a nuestra transacción.
- El método `read` itera sobre la cadena de las versiones de nuestra tupla (partiendo desde `first`) y llama el método `HeapFile::is_visible(record_header, tx_id)` cual retorna `true` si la versión de la tupla con header igual a `record_header` es visible a la transacción `tx_id`.

La función `is_visible(header, tx_id)` responde a la pregunta: *¿Debe esta versión específica ser retornada a la transacción tx_id?* El método `is_visible` funciona diferente dependiendo del nivel de aislamiento de nuestra transacción. Lo importante para notar es que para cada transacción y cada cadena de versiones de una tupla, siempre existe una única versión visible. Entonces `read` itera hasta encontrar una versión visible o hasta que llega al final de la cadena. Las reglas de visibilidad se especifican abajo:

Read Committed (RC). Una versión es visible si:

- Su **creador hizo commit** (o fue creada por esta transacción), y
- Su **borrado aún no ha hecho commit** (es decir, `t_max == 0`, o el eliminador sigue activo).

Concretamente:

```
t_min ACTIVE, t_min == tx_id:
    t_max == 0 → visible    (propia versión nueva/última)
    t_max != 0 → invisible  (propia versión reemplazada)

t_min ACTIVE, t_min != tx_id:
    → invisible            (lectura sucia bloqueada)

t_min COMMITTED:
    t_max == 0             → visible    (versión viva con commit)
    t_max ACTIVE == tx_id → invisible  (la borramos/actualizamos nosotros)
    t_max ACTIVE != tx_id → visible    (borrado sin commit de otra tx
                                     | la versión pre-borrado sobrevive)
    t_max COMMITTED       → invisible  (borrado con commit)
```

Read Committed (RC) evalúa la visibilidad de manera dinámica utilizando los estados más actualizados (*active/committed*) de `t_min` y `t_max` en cada lectura de una fila. No utiliza una instantánea (`snapshot` está vacío). Debido a esta doble comprobación dinámica, es posible que ocurran lecturas no repetibles (*non-repeatable reads*) y lecturas fantasma (*phantom reads*) si transacciones concurrentes hacen *commit* de cambios intermedios.

Snapshot Isolation (SI) - A implementar Bajo *Snapshot Isolation*, se toma un (*snapshot*) en el momento de empezar: el conjunto de todos los `tx_id` actualmente **ACTIVOS** (se almacena en `tx.snapshot`).

OJO: Presten especial atención a esta sección, ya que ustedes deberán implementar estas reglas. Una versión es visible si cumple las condiciones tanto de creador como de borrado:

Comprobación del creador (`t_min`): o bien `t_min == tx_id` (escritura de la propia transacción), o se cumplen **todas** las siguientes:

- `t_min` tiene *commit* (no se encuentra activa actualmente).
- `t_min` $\notin$ `snapshot` (no se estaba ejecutando cuando empezamos).
- `t_min` $<$ `tx_id` (comenzó antes que nosotros).

En conjunto, esto significa: *el creador hizo commit antes de que nosotros tomáramos nuestro snapshot*.

Comprobación de borrado (`t_max`): la versión sigue "viva" para nuestro *snapshot* si aplica **alguna** de las siguientes:

- `t_max == 0` — no fue borrada (retorna **true**).
- `t_max == tx_id` — la eliminó nuestra propia transacción → **invisible** (retorna **false**).
- `t_max` está actualmente ACTIVA — el borrado no tiene *commit* (retorna **true**).
- `t_max ∈ snapshot` — la transacción que realizó el borrado seguía ejecutándose cuando empezamos (retorna **true**).
- `t_max > tx_id` — la transacción que realizó el borrado comenzó después que nosotros (retorna **true**).

Si ninguna de las reglas anteriores aplica, significa que el borrado hizo *commit* antes de nuestro *snapshot* → la versión no es visible (**false**).

Actualizaciones (Updates)

Los updates son implementados en `HeapFile::update(RID rid, Record& record, TxID tx_id)`. Tanto *Read Committed* (RC) como *Snapshot Isolation* (SI) siguen exactamente los mismos pasos estructurales; la diferencia está en cómo detectan los conflictos. Efectivamente, cada actualización se encadena a la última versión de nuestra tupla en su cadena de versiones (ver Lab1).

Actualización en RC. Los pasos son los siguientes:

1. Leer el `header` del `rid`.
2. Adquirir un lock sobre `header.first` (la cabeza de la cadena de versiones).
3. Recorrer la cadena con `get_last_version(header.first)` — seguir los punteros `next` hasta que se encuentre `t_max == 0`.
4. Si la última versión está eliminada física o lógicamente, se debe retornar el centinela apropiado (por ejemplo `RID(-1, -1)` o `RID(-1, -2)`).
5. **Insertar una nueva versión** asignando `t_min = tx_id` y `first = first_rid` utilizando `insert_record`.
6. En la antigua última versión (la que acabamos de reemplazar), escribir `t_max = tx_id` y `next = new RID` utilizando `update_record_header`.

Actualización en SI - A implementar. El procedimiento es idéntico al de RC, excepto que **después de adquirir el lock**, se añade un paso de validación:

- Comprobar que la última versión (*latest version*) es **visible** para esta transacción.
- Si no es visible (es decir, otra transacción con COMMIT ya la modificó, originando un conflicto *write-write*, la transacción debe **auto-abortar** inmediatamente y lanzar una `TransactionAbortException`.
- Si la versión es visible, puede ser que fue eliminada por una transacción posterior de nosotros (`latest_header.next == RID(-1, -1)` en cual caso igual hay que hacer un abort (ver delete abajo).

La comprobación exacta es `!is_visible(latest_header) || latest_header.next == RID(-1, -1)`. Esto significa que la versión es invisible (alguien más insertó e hizo *commit* de una nueva versión después de nuestro *snapshot*) o que la cadena termina en un borrado lógico (alguien más borró la tupla).

Aborts y Undo

Una invariante importante de IIC3413DB es que cada transacción al hacer **ABORT** limpia su basura. Quiere decir que hace un **UNDO** de todos sus cambios. El flujo de cancelación (abort) de una transacción es lo siguiente:

1. Se recorre `tx.first_rids` (el conjunto de escrituras de la transacción) y llama a `heap_file->undo(first_rid, tx_id)` por cada entrada.
2. Se elimina cualquier tabla creada por esta transacción.
3. `transaction->state = COMMITTED` — lo marca como finalizado (**COMMITTED** es el estado terminal para todas las transacciones).
4. Se liberan todos los locks al final.

(A implementar): La función `HeapFile::undo(first_rid, tx_id)` debe recorrer la cadena de versiones hacia adelante empezando desde la cabeza. Utilizando `t_min` y `t_max`, nuestra transacción debe detectar la posición del primer cambio que realizó y desencadenar el resto de la cadena correctamente. Las tuplas creadas por nuestra transacción (realizando el abort) deben ser **físicamente eliminadas**, quiere decir, su `dir_slot` en la página debe ser -2. Las reglas descriptivas de que se debe hacer para cada tupla en la cadena de versiones de un mismo `first` es la siguiente:

```
t_min == tx_id:
    → borrar físicamente (dir slot = -2); continuar al siguiente 'next'

t_min != tx_id, t_max == tx_id:
    → reset_update: limpiar 't_max' y 'next', restaurando
      la versión anterior para dejarla "viva"; continuar

t_min != tx_id, t_max == 0:
    → versión viva que no es nuestra; detener (stop)
```

Esta lógica maneja tanto el *undo* de una inserción (removiendo físicamente la nueva versión) como el *undo* de una actualización o borrado (restaurando el encabezado de la versión antigua). Una vez completado el *undo*, el *heap* queda registrado como si la transacción abortada nunca se hubiera ejecutado.

Deletion

En IIC3413DB hay dos tipos de eliminación de los datos. Un **DELETE** solicitado por el usuario (cómo comando SQL) no puede remover inmediatamente los bytes del *heap* porque otras transacciones concurrentes podrían requerir leer la versión antigua — ya sea porque son transacciones RC que aún no han alcanzado esa fila, o porque son transacciones SI cuya instantánea (*snapshot*) es previa al borrado. Borrar el slot instantáneamente haría que esas lecturas devolvieran basura.

En su lugar, el borrado se registra *únicamente* como un cambio de metadatos (`t_max` actualizado y `next = RID(-1, -1)`), dejando los datos intactos. Las Reglas de Visibilidad son luego las encargadas de filtrar a cada lector: una transacción que no debería ver el borrado seguirá leyendo la versión antigua como si estuviera viva; una que sí deba ver el borrado, se encontrará con que la cadena se ha cortado abruptamente. Este es el mismo mecanismo usado para las actualizaciones — un borrado lógico simplemente es una *actualización* que no inserta ninguna nueva versión. Paso a paso, el método `HeapFile::delete_record(RID rid, TxID tx_id)` hace lo siguiente:

1. Leer el encabezado del RID proporcionado.
2. Adquirir el candado de escritura (*write lock*) sobre `header.first` – este ya está tomado por el `dleetrion` iter, pero OK.
3. Recorrer hasta llegar a la última versión disponible.
4. *Bajo Snapshot Isolation (SI)*: chequear si hay conflicto *write-write*, igual que en la actualización.
5. Llamar a `delete_record_logically(dir_slot, tx_id)`: lo cual fija `t_max = tx_id` y `next = RID(-1,-1)` en la última versión.

La versión lógica seguirá existiendo hasta que el proceso de *vacuum* la elimine estructuralmente.

El Borrado Físico solo ocurre cuando realizamos `undo` de un `ABORT` (y en *vacuum* cual no revisaremos en este lab).

Tareas para hacer en este laboratorio

Problema #1: `is_visible` [2 puntos]. Su primera tarea en este laboratorio es completar la implementación del método `is_visible` en `heap_file.cc` según la lógica explicada. En particular, deben completar la lógica de visibilidad para el nivel de aislamiento `SNAPSHOT ISOLATION`. Para hacer tests de su implementación, pueden correr el servidor de nuestro sistema y abrir dos conexiones de clientes: una bajo `RC` y otra bajo `SI`. La primera puede actualizar e insertar tuplas. Su consola `SI` no debe ver ningún cambio. También, `src/bin/test_visibility.cc` contiene una prueba con esto.

Hints: Unos hints aquí:

- `state_min`, `state_max` contienen el estado actual de las transacciones que escribieron `t_min`, `t_max`. Estos pueden ser `TransactionState::ACTIVE` o `TransactionState::COMMITTED`.
- Usando `transaction_mgr.was_active_at_snapshot(reader, writer)` pueden validar si la transacción con `TxID` igual a `writer` fue activa en el momento que la transacción `reader` empezó (i.e. está en su snapshot).

Problema #2: `undo` [1 punto]. En este ejercicio deben completar la implementación del método `undo` en `heap_file.cc` según la lógica explicada arriba. Quiere decir, partiendo desde `first` deben recorrer la cadena de versiones y hace dos cosas: (i) en la primera versión de esta cadena tocada por su transacción deben reestablecer los campos `t_max` y `next`; (ii) cada versión siguiente deben eliminar físicamente. Para tests pueden ver que pasa cuando hacen abort de una transacción y luego ven todas las tuplas de la tabla. También, `src/bin/test_undo.cc` contiene una prueba.

Hint: Dos métodos útiles:

- Para eliminar un RID físicamente, pueden llamar `page.delete_record_physically(dir_slot);`
- Para limpiar la primera versión de la tupla tocada por su transacción deben llamar: `page.reset_update(dir_slot);`

Problema #3: `delete_record` [1 punto]. En este ejercicio deben completar la implementación del método `delete_record` en `heap_file.cc` según la lógica explicada arriba. En particular, se les pide abordar el caso de eliminación bajo el nivel de aislamiento `READ COMMITTED`. Como explicado arriba, aquí deben conseguir la última versión de la tupla cual cuya cadena están modificando, validar si esta versión ya fue eliminada (sea físicamente o lógicamente) en cual caso solo retornan, y si no, eliminan la tupla lógicamente. Para conseguir la última versión de la tupla deben ocupar `get_last_version(first)`. Prueben su solución con una consola del `IIC3413DB`. También, `src/bin/test_delete.cc` contiene una prueba.

Problema #4: update [2 puntos]. En este ejercicio deben completar la implementación del método `update` en `heap_file.cc` según la lógica explicada arriba para el caso de `SNAPSHOT ISOLATION`. La lógica es la siguiente:

- Conseguir la última versión en la cadena de la tupla y su header.
- Si el header dice que la tupla fue físicamente eliminada (`is_invalid`) devolvemos `RID(-1,-2)`.
- Si el header dice que la tupla fue lógicamente eliminada *por nosotros* (`t_max==nuestro_id`), devolvemos `RID(-1,-1)`.
- Si la tupla no es visible (alguien la actualizó después de nuestro comienzo y hizo commit) hay que hacer abort.
- Si la tupla es visible, pero es eliminada (su `next==RID(-1,-1)`) hay que hacer abort.
- Si nada de esto pasa hacemos la actualización.

Para tests pueden ver que pasa cuando hacen update en una transacción y luego ven todas las tuplas de la tabla. También, `src/bin/test_update.cc` contiene una prueba.

Hint: El abort de la transacción lo realizan llamando:

```
transaction_mgr.abort_transaction(tx_id);
throw TransactionAbortException(
    "Transaction " + std::to_string(tx_id) +
    " aborted due to a write-write conflict under Snapshot Isolation."
);
```

Bug hunt [+0.2 puntos en la nota del lab]. Si encuentren un bug no-trivial en nuestra implementación y lo reportan a través de issues de *gitHub* se les asignarán una décima a la nota del laboratorio. Para este laboratorio los bugs del catalogo, o vacuum (cualquiera) no cuentan.

Código

El código base está disponible en [1] en la rama `Lab4`. Asegúrense que están usando la última versión del sistema!

Evaluación y entrega

El día límite para la entrega de esta tarea será el viernes 15 de mayo a las 23:59 horas. Para ello se utilizará el formulario de canvas. *Su entrega debe contener todo el código de la carpeta `src` y nada más, y debe compilar cuando se les agregan otras carpetas en el repositorio. El código de la carpeta `src` debe ser entregado como un archivo `.zip`.* Por último, la evaluación será en base a varios test que su solución debe responder.

Ayudantía y preguntas

Ayudantía: viernes 8 de Mayo. Preguntas: issues de github.

Referencias

[1] IIC3413DB. <https://github.com/IIC3413/2026-1>.

[2] Terence Parr. ANTLR. <https://www.antlr.org/>.